Assignment #1: Quasi-static HEV model and Rule Based Control

Table of Contents

Group information

Group number: 33

Students:

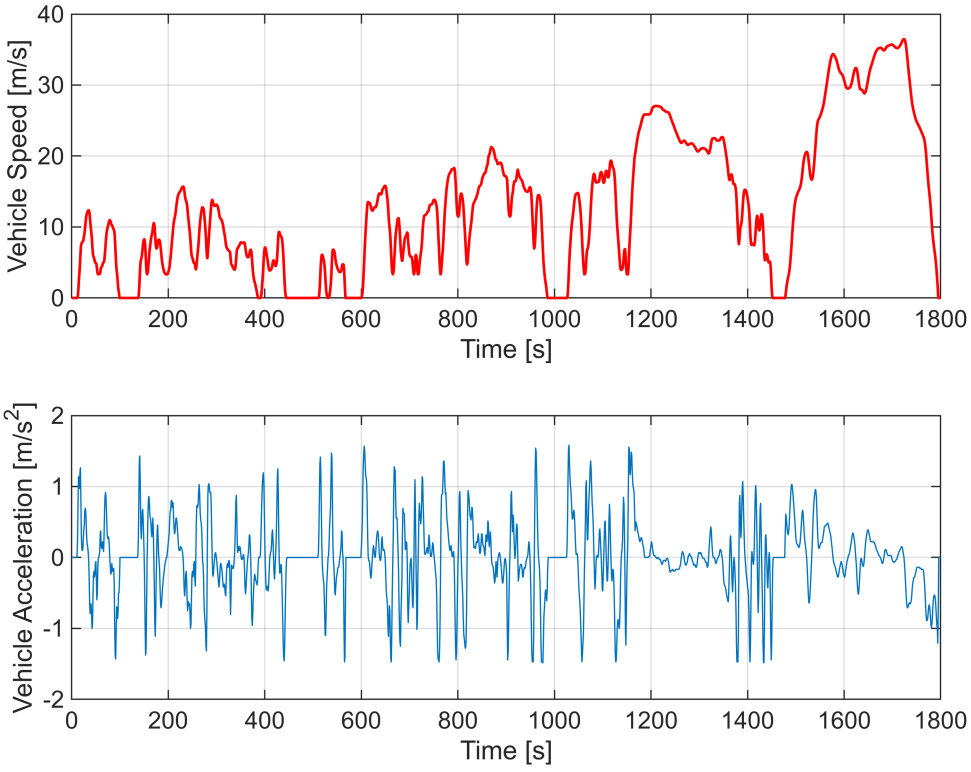
- Loris Fonseca, s342696
- Maurizio Pio Vergara, s346643
- Nikoloz Kapanadze, s342649

Load the cycle

The driving cycle to test the drivetrain performance in term of fuel consumption is the WLTP (Worldwide harmonized Light vehicles Test Procedure) which is a global driving cycle standard for determining also the fuel consumption of a vehicle. This standard cycle has been designed to be more representative of real and modern driving conditions, trying to better match the laboratory estimates of fuel consumption and emissions with the measures of an on-road driving condition. In the following section, the values of speed and acceleration at each time instant characterizing the WLTP cycle are loaded and plotted.

```
clear
close all
clc

% Load folders containing the vehicle model, mission and functions
addpath("models");
addpath("data");
addpath("utilities");
```

```
%% LOAD CYCLE DATA

mission = load("data\WLTP.mat");
time = mission.time_s;                    % [s]
vehSpd = mission.speed_km_h ./ 3.6;       % [m/s]
vehAcc = mission.acceleration_m_s2;       % [m/s^2]

%% PLOT OF WLTP CYCLE (Speed and Acceleration)

% Vehicle Speed plot
nexttile(1);
plot(time, vehSpd, Color='r', LineWidth = 1);
xlabel('Time [s]');
ylabel('Vehicle Speed [m/s]');
grid on;

% Vehicle Acceleration plot
nexttile(2);
plot(time, vehAcc);
xlabel('Time [s]');
ylabel('Vehicle Acceleration [m/s^2]');
grid on;
```



Load vehicle data

The powertrain model tested in the simulation is a series HEV, composed by a thermal engine, an eletric motor, a generator and a battery. The torque at the wheels is generated exclusively by the electric motor, which is connected to the wheels via a differential with a known transmission ratio. On the other hand, the internal combustion engine is responsible for supplying sufficient power to charge the battery when needed, through an electric generator. The simulation model of the powertrain can be described by a set of state variables, control variables, inputs and outputs. In particular, for the purpose of this simulation, i.e. the evaluation of the fuel consumption over a driving cycle, the following parameters are identifed:

- State variable: SOC (battery state of charge)
- Control variables: engine speed and engine torque
- Inputs: vehicle speed and acceleration
- Outputs: fuel consumption

```
veh = load("data\vehData.mat");

% Scale of vehicle data according to group parameters through the given scale
function
veh = scaleVehData(veh, 82000, 55000, 1260);
```

Set up our simulation loop

```
% Initialization of the state variable (SOC) and engine state (boolean vector 0=off
1=on)
% Preallocating variables

SOC = zeros(1, length(time));
engState = zeros(1, length(time));
engSpd = zeros(1, length(time));
engTrq = zeros(1, length(time));
fuelFlwRate = zeros(1, length(time));
unfeas = zeros(1, length(time));

SOC(1) = 0.6;           % the initial value of the battery SOC is assumed to be 60%
engState(1) = 0;        % the initial state of the engine is off
```

Simulation loop

The approach adopted in this simulation is the so-called backward or quasi-static approach. This energy management strategy starts from the desired output (vehicle speed and acceleration), given as an input for the control function, and works backward to determine the required inputs (fuel consumption and battery current). In the case study presented in this project, the desired output is defined by the vehicle's speed profile and acceleration, based on the standard WLTP driving cycle. The driveline model takes these parameters and determines the necessary force, and consequently the required torque, to achieve the specified conditions. It accounts for factors such as aerodynamic and rolling resistance, as well as vehicle inertia. The required torque is then used to compute the energy demand from the power sources, which in this project correspond to the internal combustion engine and the battery. Knowing the power demand, it is possible to calculate the amount

of fuel and/or battery power needed, from which the total energy consumption (fuel or battery usage) can be determined.

The following for loop implements the evaluation of the engine speed and torque and of the fuel consumption, along with the battery SOC, for each time instant of the driving cycle. In particular, the *thermostatControl* function sets the engine's operating point, while the function *hev_model* evaluates the battery SOC and fuel consumption based on the powertrain model.

```
for n = 1:length(time)
    [engSpd(n), engTrq(n)] = thermostatControl(SOC(n), engState(n), vehSpd(n),
vehAcc(n), veh);         % thermostat controller function evaluating the engine
speed and torque according to SOC, engine state and cycle point
    [SOC(n+1), fuelFlwRate(n), unfeas(n), prof(n)] = hev_model(SOC(n), [engSpd(n),
engTrq(n)], [vehSpd(n), vehAcc(n)], veh);        % function evaluating the new SOC
and the fuel consumption according to the SOC, engine operating point and cycle
point
    prof(n) = structArray2struct(prof(n));  % conversion to scalar structures
containing arrays
    engState(n+1) = engTrq(n) > 0;          % engine state update checking the
engine torque value
end
```

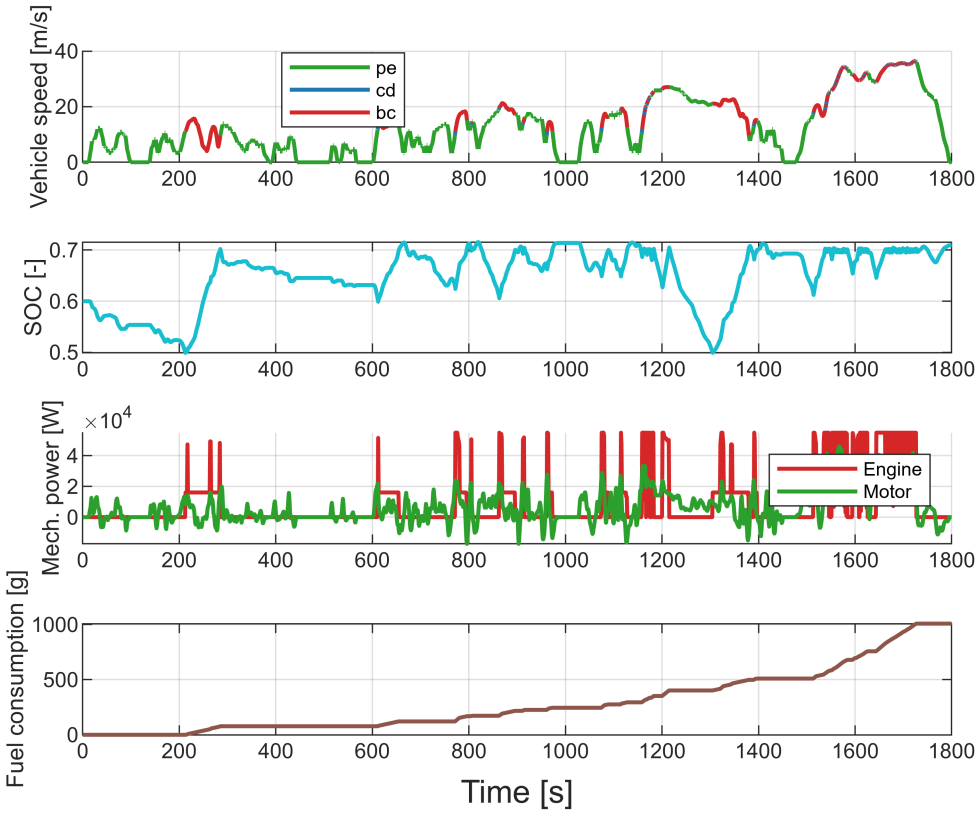
Results analysis

The key results obtained are summarized below. They illustrate, in sequence, the drivetrain's operating modes according to the vehicle speed profile, the battery State Of Charge, the comparison between engine and motor power, and the fuel consumption, all presented as functions of time.

By analyzing the first plot, the different operating modes of the hybrid powertrain can be identified. The first mode is the pure electric mode (pe), in which the engine is off. In this condition, the battery discharges when the required power at the wheels is positive and charges when the motor operates in regenerative braking. The second mode is the charge-depleting mode (cd), where the engine is running but the battery continues to discharge because the power provided by the engine is lower than the power delivered by the battery to the electric motor. The last mode is the battery charging mode (bc), in which the engine is on and actively charging the battery. As a result, the SOC exhibits a steep increase, unlike in regenerative braking with engine off, where the current delivered to the battery is lower, leading to a smoother SOC rise. Whenever the vehicle operates in battery charging mode, the speed profile is marked by red segments, always corresponding to this steep SOC increase, verifying the correlation between SOC variation and speed profile.

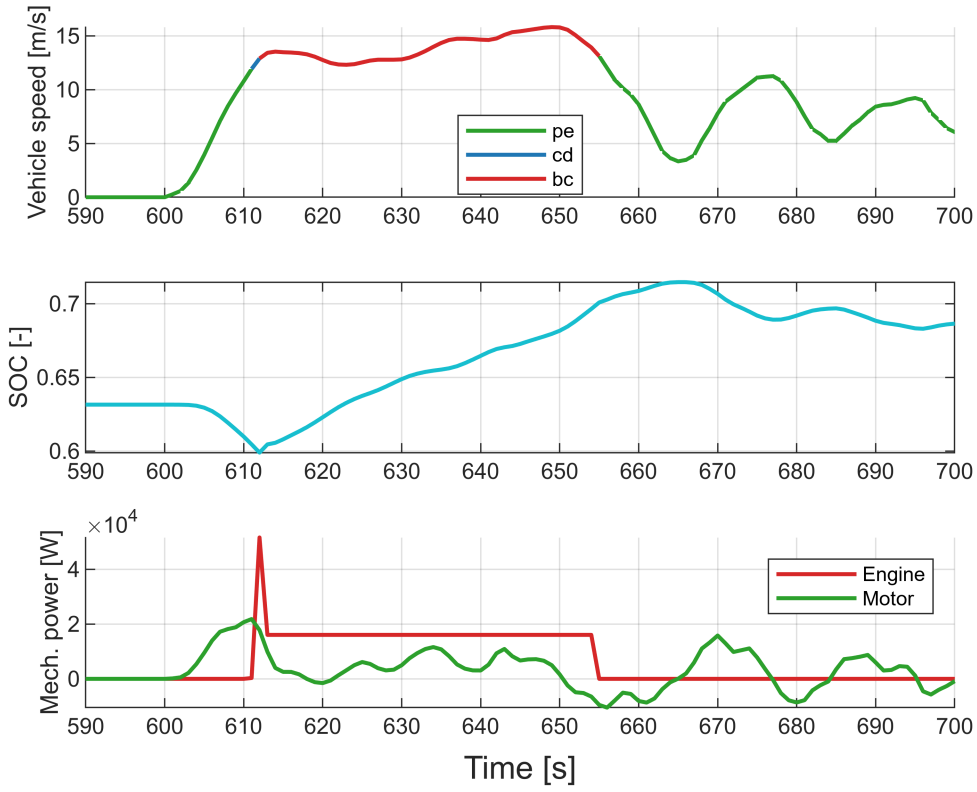
Another important observation is that the engine sometimes turns on and charges the battery even when the SOC has not yet reached its lower limit, for instance around time instant 600. This occurs due to the thermostat control logic, which regulates battery power. Specifically, if the engine is off but the power demand from the battery to the electric motor exceeds the battery's maximum power limit, the engine is activated to supply the excess power. Subsequently, the engine remains on until the battery is recharged up to 70% limit, even if in the following time instants the battery alone could meet the motor's power demand.

```
% MAIN PROFILES PLOT
[fig, t] = mainProfiles(prof);
```



In the following plot, the previously discussed statement about battery limits can be observed. Additionally, a peak in SOC exceeding the upper limit is noticeable. This occurs due to regenerative braking, since vehicle acceleration is negative, which continues to charge the battery even when it has already reached its upper threshold. This condition persists until approximately t = 665, when the SOC reaches its maximum, the vehicle speed reaches a local minimum, and acceleration drops to zero. At this point, it can also be seen that the engine is correctly turned off, confirming the expected behavior of the control system.

```
% Plot illustrating overcharging caused by regenerative braking
[fig3, t3] = overcharging(prof);
```

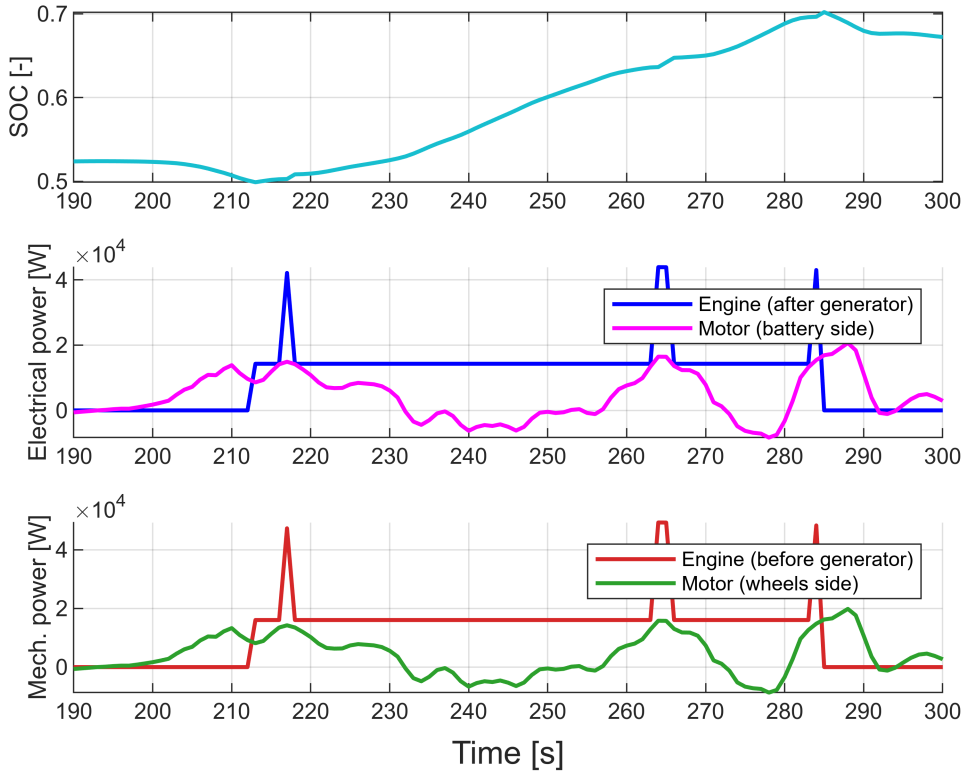


Another key aspect is the comparison between engine and motor power, which helps validate the thermostat control logic. As explained later in the thermostat control logic section, when the engine is turned on to charge the battery, its operating point is set to ensure that the power delivered to the battery is always greater than the power required by the electric motor. This prevents unintended battery discharge during charging. This behavior is particularly evident at low loads when the electric motor power does not exceed the battery power limits, allowing the engine to remain off without needing to compensate for a power deficit, hence ensuring that the engine operating point is determined only according to the previously explained condition. For example, in the analyzed time range of the following chart, this behavior is evident. When the SOC falls below the minimum threshold and the engine is switched on, the blue engine profile assumes two distinct values: one corresponding to the optimal engine operating point and another to the maximum allowed power output. Whenever motor power, represented by the magenta profile, exceeds the optimal engine power, the control system sets the engine to its maximum allowed power. This scenario occurs three times in the observed time window, as indicated by the three peaks in engine power.

It is important to note that, unlike in previous chart, the power profiles used for this comparison differ from those previously displayed, and here reported again as third chart. The comparison between green and red profile is based on the mechanical power of both the engine and motor, without accounting for efficiency losses. However, the thermostat control logic compares power at the generator output and motor input, i.e., electrical power, taking into account motor and generator efficiencies. These losses reduce the effective engine power while increasing the required motor power during motoring, which must be considered for a proper assessment of the controller's performance. Considering efficiency is crucial when evaluating the controller's performance: if the mechanical power chart is used for validation, the results might appear incorrect, as the motor power never exceeds the engine power in this comparisons. In contrast, the electrical power comparison charts show

that the motor power can indeed surpass the engine power due to efficiency losses, providing a more accurate representation of the system's behavior.

```
% Plot for electrical-mechanical power comparison
[fig1, t1] = electricalPowerComparison(prof);
```



As previously mentioned, when the required battery power limit is reached, the engine is activated to compensate for the power deficit. This situation typically occurs at high speeds, where the power demanded by the electric motor, and consequently supplied by the battery, is significant, even if the battery SOC is close to the 70% threshold. The following charts analyze a segment of the driving cycle where this condition occurs.

In the initial displayed time interval, the thermostat control operates in the "standard" mode, charging the battery up to its upper limit. Due to the high power demand, the engine is set to its maximum power output. Once the SOC reaches the upper threshold, the engine would normally turn off. However, since the power required by the electric motor exceeds the maximum power the battery can supply, the engine remains on, adjusting its operating point to compensate for the power gap between the motor and battery output.

The validity of the controller under these conditions can be assessed by examining the difference between the engine and motor electrical power at each time step. This difference should always match the maximum battery power for a SOC of 70%. For example, at time instant t = 1679, the recorded electrical power values are:

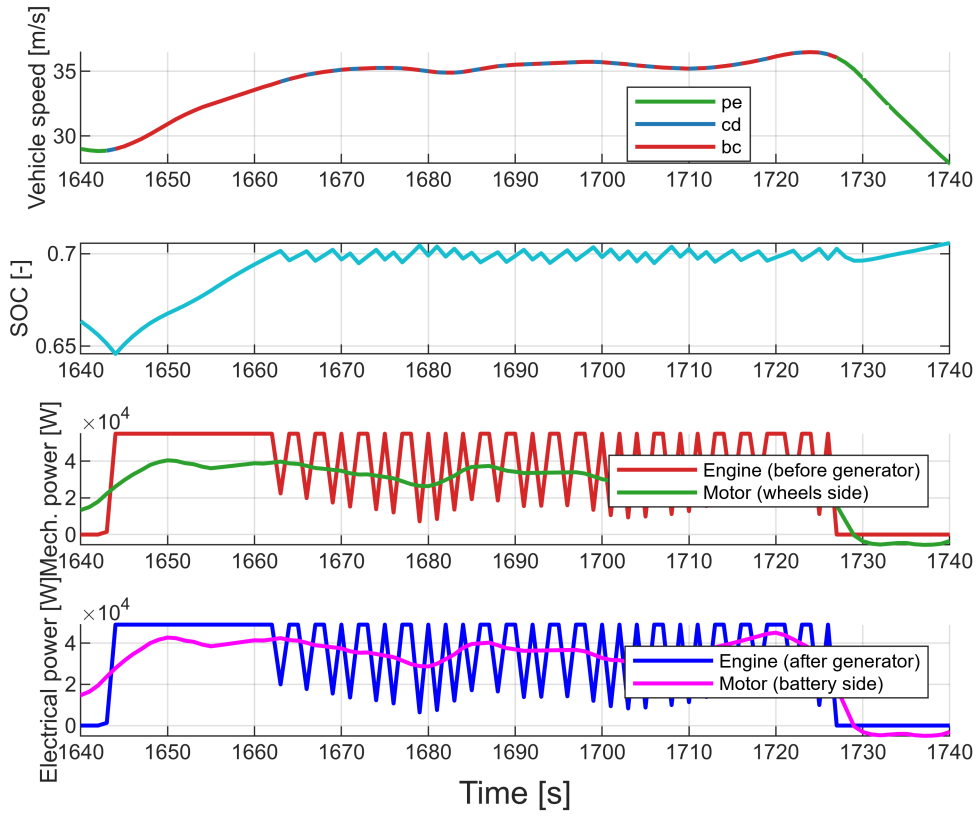
- **Motor power**: *p_motor* = 28.844 kW
- **Engine power**: *p_engine* = 6.408 kW
- **Power difference**: *p_motor* - *p_engine* = 22.436 kW

This last value precisely matches the maximum battery power at SOC = 0.7, confirming the controller's correct operation.

In the subsequent time step, the engine remains on because the SOC is still below 70%, requiring additional engine power to charge the battery. As mentioned earlier, under high power demand conditions, the engine operates at its maximum power. Once the SOC reaches 70%, the engine is then used solely to offset the power deficit.

This high power demand scenario causes the drivetrain's operating modes to fluctuate between battery charging and charge depletion, corresponding to the red and blue lines in the vehicle speed profile, respectively, as the engine power oscillates around the motor power profile, leading to a cyclical charging-discharging pattern. This oscillatory behavior ceases when the vehicle begins to decelerate (around t = 1730), transitioning into pure electric mode with regenerative braking, which increases the SOC over the upper limit. At this point, the engine is finally turned off.

```
% Plot for high speed behaviour
[fig2, t2] = highSpeedComparison(prof);
```

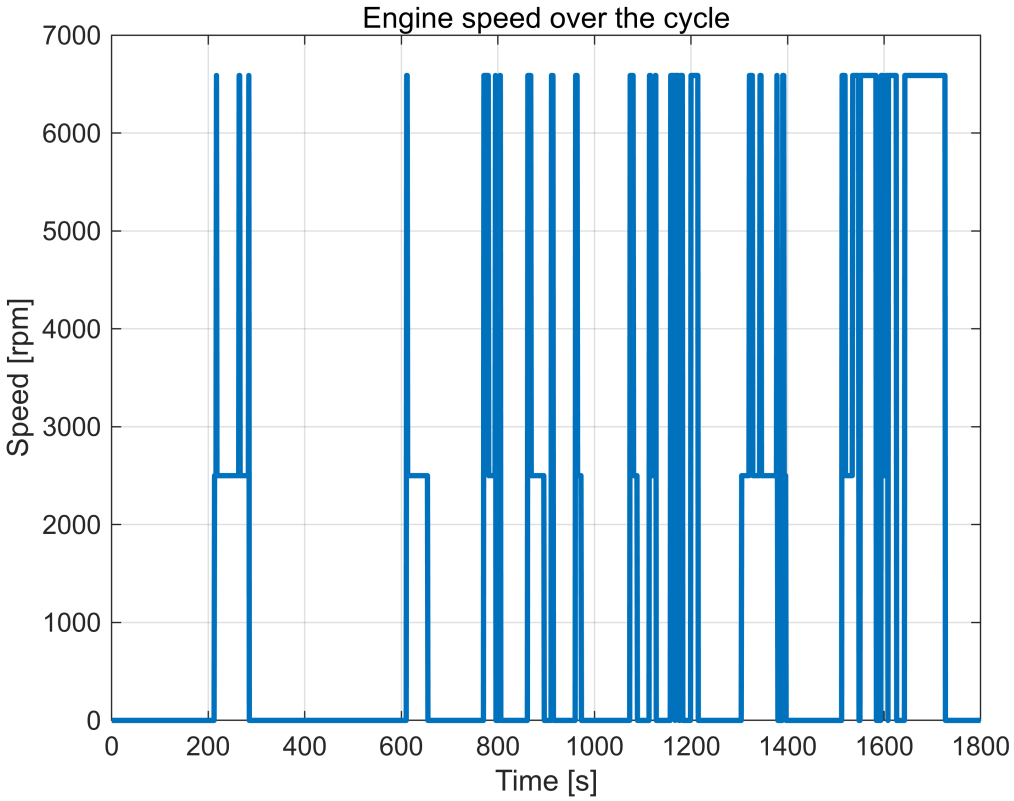


A further validation of the controller's functionality can be performed by analyzing the engine's speed and torque profiles. As explained in the dedicated controller section, the engine speed is set to only two values: one corresponding to optimal engine power and the other to maximum engine power. As shown in the following chart, the engine speed consistently assumes only these two values, confirming the controller's effectiveness.

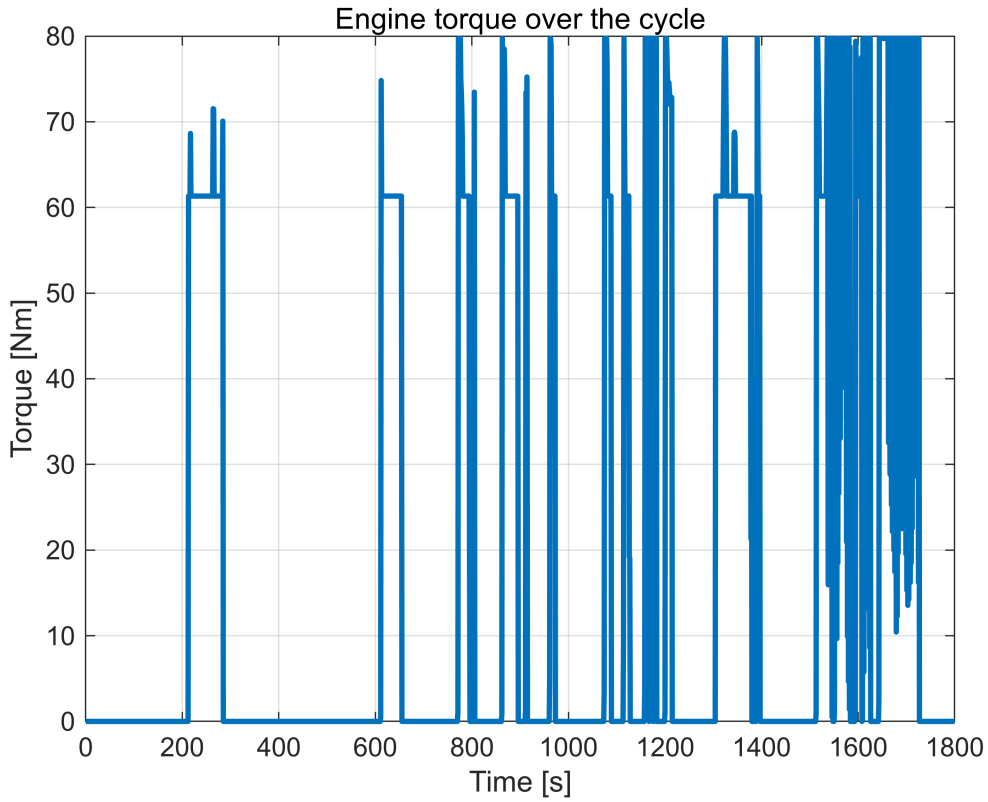
In contrast, the torque profile behaves differently. Theoretically, it should also assume only two values. However, power saturation is managed by maintaining a constant speed while adjusting the engine torque. As a result,

the torque profile exhibits multiple values. This plot is particularly useful for identifying the exact moments when battery power saturation occurs. Specifically, if the torque at a given instant deviates from the values corresponding to maximum or optimal power, it indicates that saturation is occurring at that moment.

```
% Engine speed plot
figure;
plot(time, engSpd * 30/pi, LineWidth=2);
xlabel('Time [s]')
ylabel('Speed [rpm]');
title('Engine speed over the cycle');
grid on;
```




```
% Engine torque plot
figure;
plot(time, engTrq, LineWidth=2);
xlabel('Time [s]')
ylabel('Torque [Nm]');
title('Engine torque over the cycle');
grid on;
```

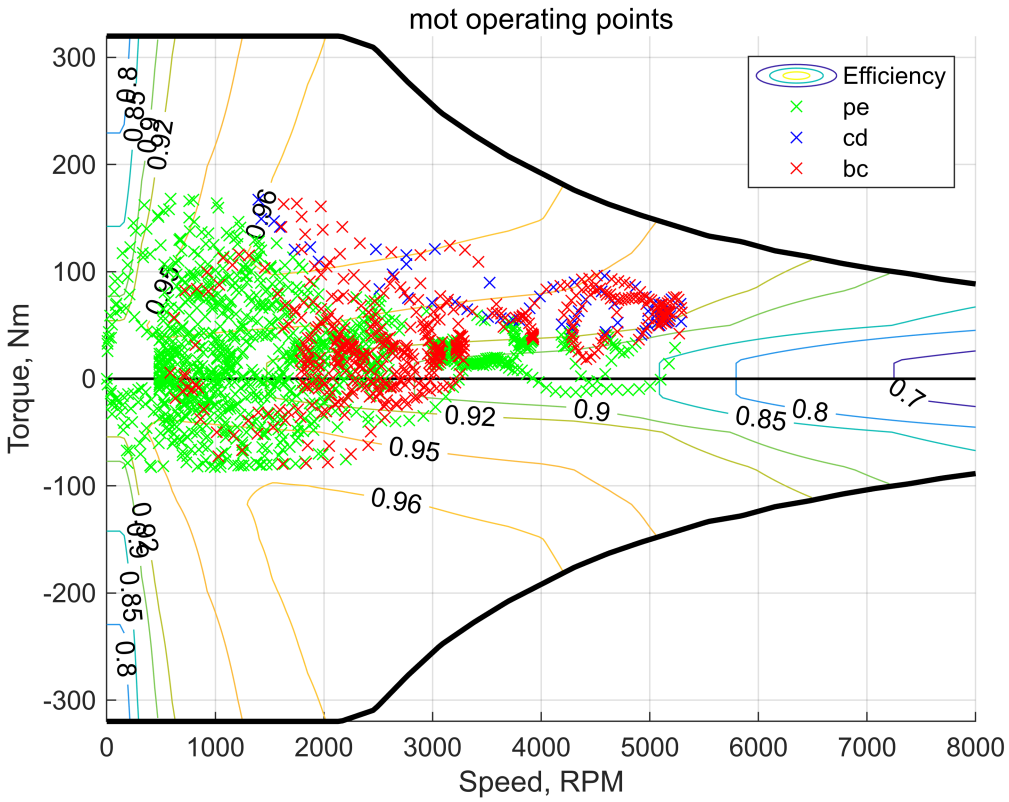


The following charts depict the motor, engine and generator operating points on their corresponding maps. Again, these tools can help to assess the correct functioning of the controller, but their main goal is the evaluation of the efficiencies of the differrent elements according to their operation points which are essential for the power balance equations.

The motor map represents torque as a function of motor speed. As is typical for electric motors, this relationship features a constant torque profile at lower speeds and a constant power profile at higher speeds, which results in a hyperbolic torque curve. In addition to the torque profile, the map also includes efficiency isolines across the entire operating range of the electric motor. It can be noted that the high efficiency zone extends along a wide area of the motor operating zone. Notably, since this type of machine can generate both positive and negative torque, its operational area extends in both directions. Furthermore, the positive and negative torque profiles are almost symmetrical with respect to the x-axis.

It can be noticed that the electric motor operating points extend in a wide operating zone. This beacause the electric motor operating point cannot be directly controlled, since it depends on the vehicle speed and acceleration, and so on the driving mission, being the motor mechanically connected to the wheels. Another consideration that can be done is that the motor operating points never go close to the upper and lower limits, and never go out from the feasible operating area. This mean that the motor is oversized for the considered driving mission. Therefore, in order to reduce the cost and the required space for the installation, a smaller electric motor could be chosen.

```
% Electric motor map operating points
emMapWithPF(veh.mot, prof, 'mot', 'all');
```

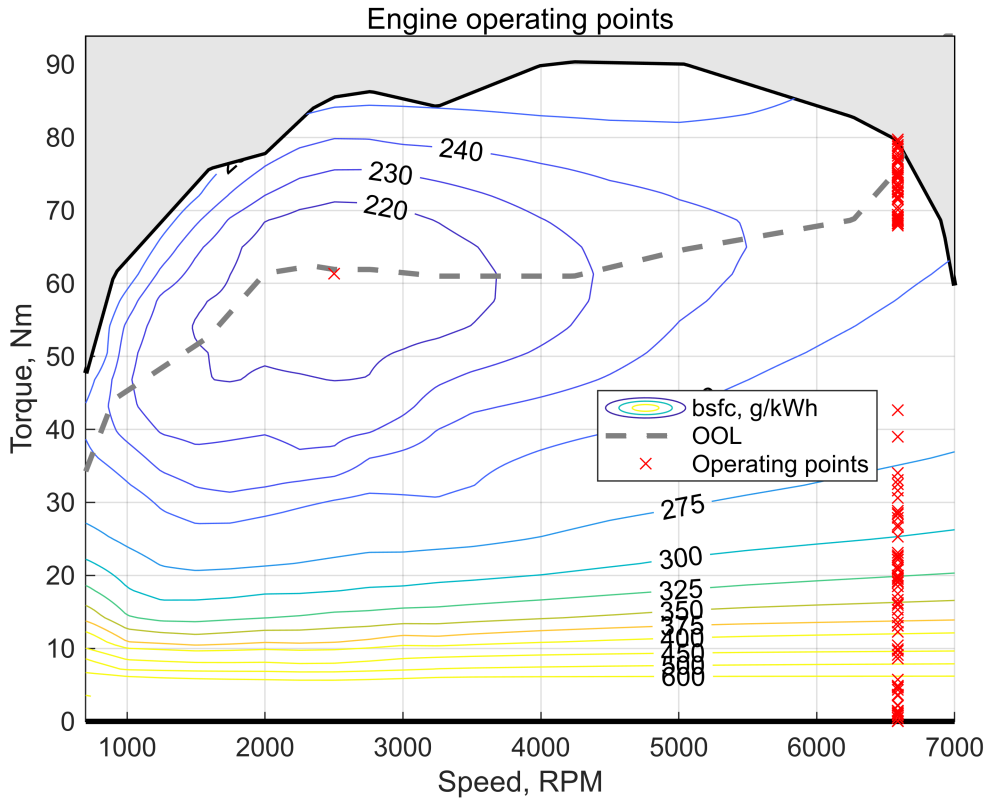


The engine map represents engine effciency, expressed as brake specific fuel consumption, as a function of engine speed and torque. It is important to note that, differently from the electric motor, the speed-torque relationship of an internal combustion engine has a more complex shape, with peak torque occurring at mid-range speeds. Additionally, the highest efficiency zone, corresponding to the minimum brake specific fuel consumption, covers a much smaller area and is located at low-mid speeds under relatively high loads. Alongside the *bsfc* map, the Optimal Operating Line (OOL) is also plotted, indicating the operating points with the lowest *bsfc* for each engine speed.

The obtained operating points confirm compliance with the control strategy, as they are distributed across only two speed values. When operating at the speed corresponding to maximum power, different torque values are observed, aligning with the earlier considerations on engine speed and torque profile plots.

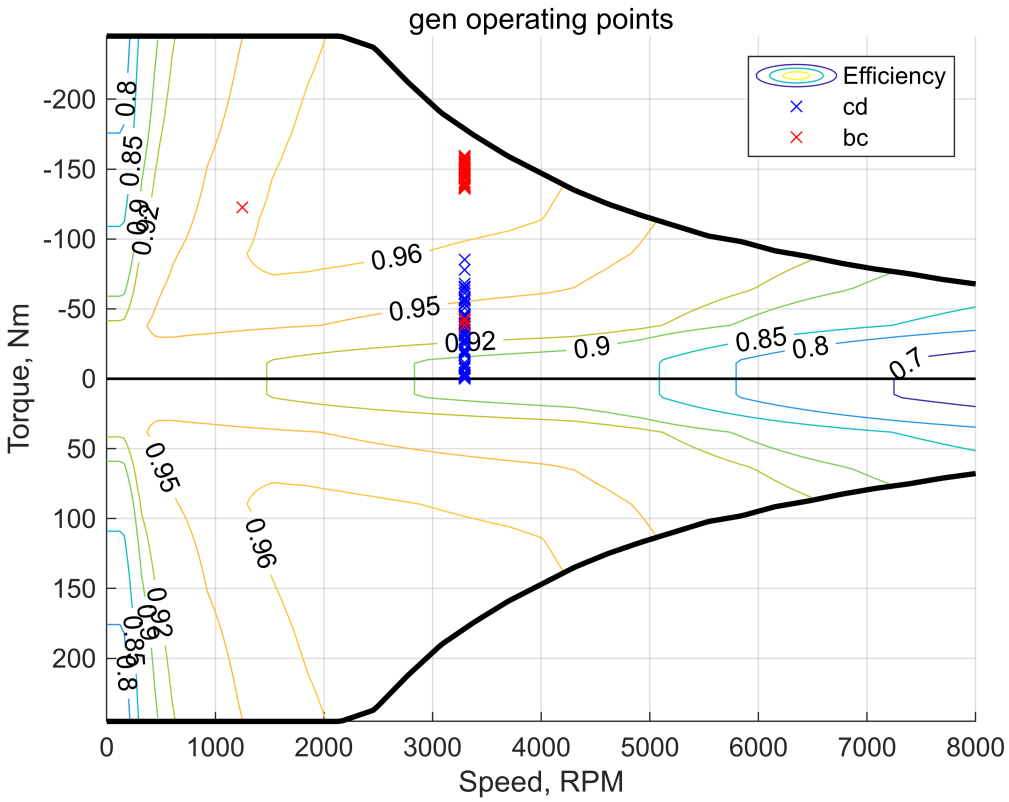
An additional insight provided by this analysis is fuel consumption. It is evident that when the engine operates at either the optimal or maximum power points, fuel consumption is minimized, as these points lie on the OOL. However, when saturation occurs, efficiency decreases significantly, leading to higher brake specific fuel consumption values. Therefore, an optimization strategy could involve adjusting the engine's operating point to remain on the OOL even during saturation, thereby improving overall efficiency.

```
% Engine map operating points
engMapWithPF(veh.eng, prof, 'bsfc');
```



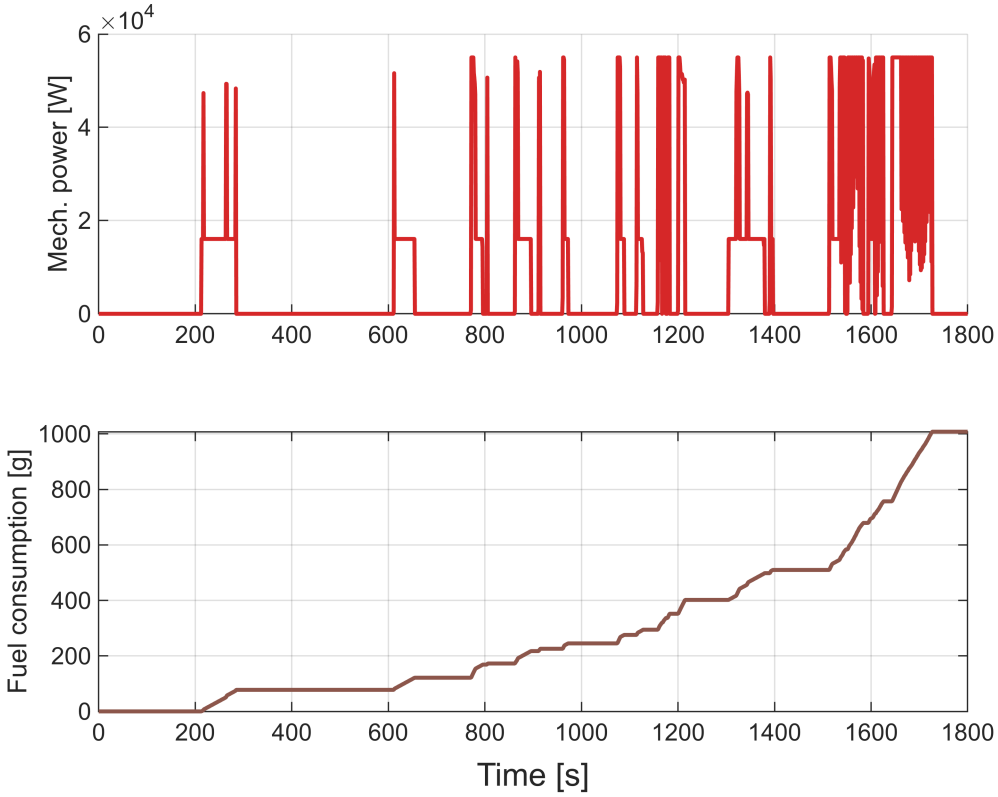
Similarly to the electric motor, the generator map can also be plotted, showing a comparable trend. Since the engine and generator are mechanically connected through an intermediate gear, the generator's speed and torque can be determined by simple mathematical relationships. Specifically, the absolute values of the generator's speed and torque are obtained by dividing and multiplying the engine speed and torque, respectively, by the transmission ratio. Given that the transmission ratio is 0.5, the generator speed is half of the engine speed, while the torque is doubled.

```
% Generator map operating points
emMapWithPF(veh.gen, prof, 'gen', 'all');
```



As previously mentioned, once the engine operating points are known, it is possible to plot and evaluate fuel consumption throughout the cycle. The following charts illustrates the evolution of fuel consumption over time and the engine power. As expected, fuel consumption remains constant when the engine is off and increases when it is on. Notably, the fuel consumption profile exhibits a parabolic trend, as higher speeds toward the end of the cycle demand more power from the engine.

```
% Fuel consumption-engine power relation
[fig4, t4] = fuelConsumption(prof);
```



The total fuel consumption is calculated by integrating the fuel flow rate obtained from the `hev_model` function during the simulation loop, using the cumulative integral function `cumtrapz`. Additionally, the cycle distance is determined by integrating speed over time. Consequently, fuel economy can be computed by incorporating fuel density into the calculations.

```
fuelConsumption = cumtrapz(time, fuelFlwRate);                                  %
integral of the fuel flow rate over the time [g]
fuelConsumption = fuelConsumption(length(time)) * 10^(-3);                      %
total fuel consumption is the last value of the cumulative integration [kg]

cycleDistance = trapz(time, vehSpd) / 10^3;                                     %
[km]
fuelEconomy = fuelConsumption / veh.eng.fuelDensity * 100/cycleDistance;        %
[L/100km]
finalSOC = SOC(length(time));                                                   %
final SOC is the last value of the SOC profile [-]
```

Save results

```
% Store results
save("results.mat", "prof", "fuelConsumption", "fuelEconomy", "finalSOC")
```

- `prof` is the profiles structure created in this section.
- `fuelConsumption` is the total fuel consumption for the whole mission (in kg).

- `fuelEconomy` is the distance-specific fuel consumption for the whole mission (in L/100km).
- `finalSOC` is the final battery SOC.

Improving the controller

The thermostat controller operates as requested; however, as mentioned before, it has some limitations due to the fact that it only features two engine speed operating points. This results in higher fuel consumption and may reduce driving comfort at high vehicle speeds due to the constant engine noise at maximum speed. A more effective approach would be to vary the engine speed and torque according to the Optimal Operating Line (OOL) while still meeting power demand. This would lead to lower fuel consumption.

The general setup and simulation loop remain the same as before. The key difference lies in the thermostat control function, which is designed to achieve a lower fuel economy by prioritizing a more efficient operating strategy. To avoid overwriting previous variables, all new variables are renamed with the suffix "_imp".

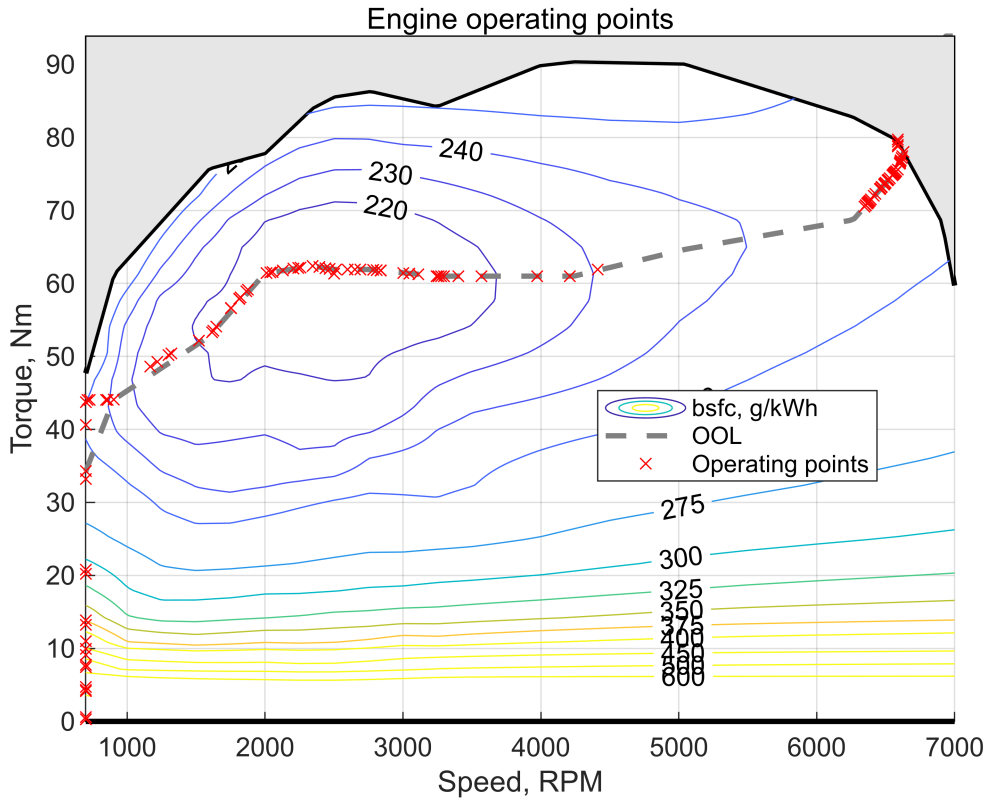
The results are saved in the file *results_extra.mat*, where it is possible to observe a 3 to 4% reduction in fuel economy. This reduction aligns with the goal of optimizing engine operation by moving towards lower brake-specific fuel consumption values, ultimately leading to more efficient fuel usage and a nominal decrease in overall fuel economy.

```
% _imp will be used for the improved controller variables
SOC_imp = zeros(1, length(time));
engState_imp = zeros(1, length(time));
engSpd_imp = zeros(1, length(time));
engTrq_imp = zeros(1, length(time));
fuelFlwRate_imp = zeros(1, length(time));
unfeas_imp = zeros(1, length(time));

SOC_imp(1) = 0.6;           % the initial value of the battery SOC is assumed to be
60%
engState_imp(1) = 0;        % the initial state of the engine is off

for n = 1:length(time)
    [engSpd_imp(n), engTrq_imp(n)] = thermostatControl_improved(SOC_imp(n),
engState_imp(n), vehSpd(n), vehAcc(n), veh);
    [SOC_imp(n+1), fuelFlwRate_imp(n), unfeas_imp(n), prof_imp(n)] =
hev_model(SOC_imp(n), [engSpd_imp(n), engTrq_imp(n)], [vehSpd(n), vehAcc(n)], veh);
    prof_imp(n) = structArray2struct(prof_imp(n));  % conversion to scalar
structures containing arrays
    engState_imp(n+1) = engTrq_imp(n) > 0;          % engine state update checking
the engine torque value
end
```

From the plot below, it is possible to observe that the operating points are distributed along the Optimal Operating Line (OOL). However, some points at very low speed and torque, resulting from a low engine power command, remain in the area of high *bsfc*. This occurs because, at very low power demand, the engine speed

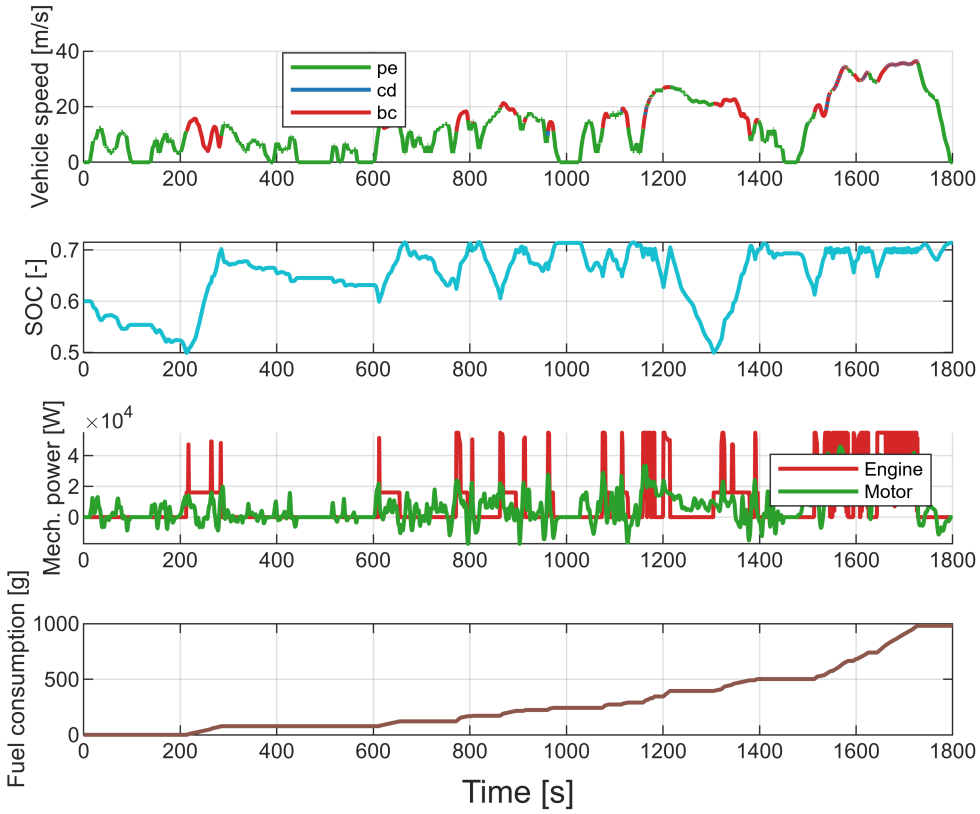
cannot be reduced below the idle speed. As a result, the engine operates at very low torque values, leading to lower efficiency. Therefore, while this strategy improves overall efficiency by keeping the engine closer to the OOL, low-load conditions still result in less favorable fuel consumption.

```
% Engine map operating points - improved
engMapWithPF(veh.eng, prof_imp, 'bsfc');
```



From the plot below, it is possible to observe that the general behavior of the drivetrain modes and the SOC charge/discharge dynamics remain similar to the previous case. The same trend for the engine power command and SOC at high vehicle speeds can be seen in the final part of the WLTP cycle. Additionally, the battery recharge process, which starts early at 60% due to power saturation and keeps the engine running until the SOC reaches its upper limit again, follows the same pattern observed with the previous energy management function.

```
% MAIN PROFILES PLOT
[fig_imp, t_imp] = mainProfiles(prof_imp);
```




```
% Evaluate fuel consumption, fuel economy and final SOC
fuelConsumption_imp = cumtrapz(time,
fuelFlwRate_imp);                                    % integral of the fuel flow rate
over the time [g]
fuelConsumption_imp = fuelConsumption_imp(length(time)) *
10^(-3);                        % [kg]

fuelEconomy_imp = fuelConsumption_imp / veh.eng.fuelDensity * 100/
cycleDistance;          % [L/100km]
finalSOC_imp =
SOC_imp(length(time));                                                    % [-]

% Store results
save("results_extra.mat", "prof_imp", "fuelConsumption_imp", "fuelEconomy_imp",
"finalSOC_imp")
```

The thermostat controller

Once the powertrain model has been established, the next step is to develop an energy management strategy by implementing a rule-based control approach, specifically the thermostat control. The primary objective of this control strategy is to determine the engine operating point, defined by speed and torque, at each time instant of the given vehicle cycle. The engine operating conditions are primarily determined by two key factors: the battery's State Of Charge and the maximum and minimum power the battery can supply. To implement the control strategy within the controller, the following four sequential steps are followed:

1. Definition of the engine state
2. Determination of the engine power
3. Power saturation if battery limits are exceeded
4. Definition of engine speed and torque according to the determined engine power

```
function [engSpd, engTrq] = thermostatControl(SOC, engState, vehSpd, vehAcc, veh)
```

Phase 1: check if SOC is within the limits

The first step in the thermostat control strategy is defining the engine state, which determines whether the engine is switched on or off. In this project, the engine state is managed based on the battery's State of Charge. The objective is to maintain the SOC within predefined upper and lower limits. Specifically, the engine is activated when the SOC drops below the minimum threshold to recharge the battery, and it is deactivated when the SOC exceeds the maximum threshold. The SOC limits defined for this project are the following:

- SOCmax = 70%
- SOCmin = 50%

```
% Definition of the SOC thresholds (max and min)
SOCmax = 0.7;
SOCmin = 0.5;

if SOC >= SOCmax
    engState = 0;       % turn off the engine if SOC exceeds the maximum limit
elseif SOC <= SOCmin
    engState = 1;       % turn on the engine if SOC goes below the minimum limit
end                     % if SOC is between the limits, keep the engine state
```

Phase 2: firstly assess with hev_drivetrain function the demand torque and demand speed, then evaluate the demand power

Once the engine state is determined, the next step is to define the engine power. In this simulation, only three distinct power levels are considered based on the operating conditions. If the engine is off, its power is naturally set to zero. When the engine is on, two possible scenarios arise. The first occurs when the required power at the wheels is relatively low, allowing the engine to operate at an optimal power level, identified in this simulation as the point where brake specific fuel consumption is minimum. The second scenario arises when the power demand at the wheels is high, making the optimal engine power insufficient to charge the battery, as the power required by the electric motor exceeds the amount provided by the engine at its optimal operating point. In this case, the engine power is set to its maximum possible value to ensure that sufficient energy is generated to recharge the battery. To summarize:

- **Engine OFF:** engine power set to zero
- **Engine ON:** engine power se to optimal operating point (low power demand at wheels) or maximum operating point (high power demand at wheels)

```
[shaftSpeed, shaftTorque, ~] = hev_drivetrain(vehSpd, vehAcc, veh);     % required
shaft speed and torque of the e-motor
```

```
shaftPower = shaftTorque * shaftSpeed;                                    % [W] power
at e-motor shaft

% Evaluation of electric motor efficiency corresponding to the motor operating
point through motor map
motSpeed = veh.mot.effMap.GridVectors{1, 1} * 30/pi;      % [rpm]
motTorque = veh.mot.effMap.GridVectors{1, 2};             % [Nm]
motEfficiency = veh.mot.effMap.Values;                    % [-]
motEfficiencyOP = interp2(motSpeed, motTorque, motEfficiency', shaftSpeed * 30/pi,
shaftTorque, 'linear');       % motor efficiency value corresponding to shaft speed
and torque

% Computation of the power demand, i.e. the electric power at motor input (battery
side)
if shaftTorque >= 0
    motPower = shaftPower * 1/motEfficiencyOP;        % if vehicle accelerates, the
motor works in motoring
else
    motPower = shaftPower * motEfficiencyOP;          % if vehicle brakes, the
motor works in regenerative braking
end

% Definition of optimal and maximum engine power operating points -> optimal engine
operating point = min bsfc (max efficiency), maximum power operating point = max
power

% Optimal engine power
[~, minbsfc_index] = min(veh.eng.bsfcMap.Values(:));
[rowMinbsfc, colMinbsfc] = ind2sub(size(veh.eng.bsfcMap.Values),
minbsfc_index);              % row and column indexes corresponding to min bsfc

engTorqueOpt = veh.eng.bsfcMap.GridVectors{1,2}(colMinbsfc);              %
[Nm]      % engine torque corresponding to min bsfc
engSpeedOpt = veh.eng.bsfcMap.GridVectors{1,1}(rowMinbsfc) * 30/pi;       %
[rpm]     % engine speed corresponding to min bsfc
engPowerOpt = engTorqueOpt * engSpeedOpt * pi/30;                         %
[W]       % optimal engine power corresponding to min bsfc

% Maximum engine power
engSpeed = veh.eng.maxTrq.GridVectors{1,1} * 30/pi;         % [rpm]
engTorque = veh.eng.maxTrq.Values;                          % [Nm]
engPower = engTorque .* engSpeed * pi/30;                   % [W]       % maximum
power profile

[engPowerMax, engPowerMax_index] = max(engPower);           % [max power value,
index corresponding to max power]
engSpeedMaxPower = engSpeed(engPowerMax_index, 1);          % [rpm]     % engine
speed corresponding to max power
engTorqueMaxPower = engTorque(engPowerMax_index, 1);        % [Nm]      % engine
torque corresponding to max power
```

```
effGen = 0.89;  % we cannot evaluate the efficiency in advance without knowing the
engine operating point and so the generator speed and torque
                % we set the generator efficiency at 0.89 to be more conservative
because during the cycle it happens that it goes below 0.9

if engState
   if  motPower <= effGen * engPowerOpt
       engPowerCmd = engPowerOpt;           % if engine is ON AND the optimal
engine power is sufficient to charge the battery, the engine power is set to the
optimal value
   else
       engPowerCmd = engPowerMax;           % if engine is ON BUT the optimal
engine power is NOT sufficient to charge the battery, the engine power is set to
the max value
   end
else
    engPowerCmd = 0;                        % if engine is OFF, required engine
power set to 0
end
```

Phase 3: add limits on the battery power

After computing the engine power according to the engine state and the demand power at the wheels, a power saturation must be implemented if the battery power limits are exceeded. Indeed, the modeled battery is characterized by some constraints, either in term of power or current. Therefore, the engine power must be corrected if the battery power limits are not respected. To evaluate the amount of engine saturation, if needed, the power balance at battery DC bus must be considered. The battery power is equal to the demand power plus the one provided by the electric generator. In this project, the used convention declares that the power exting from the battery is positive, as well as the current, meanwhile the current entering in the battery is negative. Therefore, being the power of the generator a charging power, i.e. entering in the battery, the power balance equation can be rewritten considering the electric generator power as the engine power multiplied by the engine efficiency, but with the opposite sign.

The limits to be implemented in the controller are three, and are listed below:

- **Constraint 1:** battery power lower than the maximum allowed value
- **Constraint 2:** battery power higher than the minimum allowed value
- **Constraint 3:** avoid negative engine power (unfeasible) when the power generated with regenerative braking exceeds the battery limit

```
% evaluation of the maximum and minimum battery power for the considered SOC
battPowerMax = interp1(veh.batt.maxPwr.GridVectors{1,1}, veh.batt.maxPwr.Values,
SOC, 'linear');        % interpolation between the given SOC-MaxPower points
battPowerMin = interp1(veh.batt.minPwr.GridVectors{1,1}, veh.batt.minPwr.Values,
SOC, 'linear');        % interpolation between the given SOC-MinPower points

engPowerCmd = max(engPowerCmd, (motPower - battPowerMax)/effGen);       %
implementation of first constraint
```

```
engPowerCmd = min(engPowerCmd, (motPower - battPowerMin)/effGen);       %
implementation of second constraint

engPowerCmd = max(engPowerCmd, 0);                                      %
implementation of third constraint
```

Phase 4: choose engine speed and engine torque

Once the engine power is determined based on the cycle point, SOC, and battery power saturation, the corresponding engine speed and torque must be set to achieve the required power. If the engine power is set to the optimal value, both speed and torque adjust accordingly. Similarly, if the command corresponds to maximum power, speed and torque are set to the values that deliver maximum power. When the engine power command is zero, the engine remains off, resulting in both speed and torque being zero. Lastly, if saturation leads to an intermediate engine power command, different from zero, the optimal value, or the maximum value, the speed remains constant at its maximum value, while the torque is calculated as the ratio of the engine power command to the engine speed.

```
if engPowerCmd == engPowerOpt            % speed and torque values at optimal
engine power
    engSpd = engSpeedOpt * pi/30;        % [rad/s]
    engTrq = engTorqueOpt;               % [Nm]

elseif engPowerCmd == engPowerMax        % speed and torque values at max engine
power
    engSpd = engSpeedMaxPower * pi/30;   % [rad/s]
    engTrq = engTorqueMaxPower;          % [Nm]

elseif engPowerCmd == 0                  % speed and torque values at zero engine
power
    engSpd = 0;
    engTrq = 0;
else                                     % if engine power command is different
from zero, the optimal value, or the maximum value due to saturation, set the
engine speed corresponding to the max power and compute the torque
    engSpd = engSpeedMaxPower * pi/30;   % [rad/s]
    engTrq = engPowerCmd/engSpd;         % [Nm]
end

end
```

A variant of the thermostat controller

Another approach to setting the engine speed and torque can be considered to improve fuel economy. The idea is to remain on the Optimal Operating Line (OOL) when the power command is different from both the optimal and maximum values, while still respecting the torque limit. By doing so, the engine operating points that lie in low-efficiency zones of the engine map are minimized, resulting in improved fuel consumption throughout the driving cycle.

This process is thoroughly explained in *Phase 4* of the function code, whereas the first three phases remain unchanged from the previous function.

```
function [engSpd, engTrq] = thermostatControl_improved(SOC, engState, vehSpd,
vehAcc, veh)
```

Phase 1: check if SOC is within the limits

```
% Definition of the SOC thresholds (max and min)
SOCmax = 0.7;
SOCmin = 0.5;

if SOC >= SOCmax
    engState = 0;       % turn off the engine if SOC exceeds the maximum limit
elseif SOC <= SOCmin
    engState = 1;       % turn on the engine if SOC goes below the minimum limit
end                     % if SOC is between the limits, keep the engine state
```

Phase 2: firstly assess with hev_drivetrain function the demand torque and demand speed, then evaluate the demand power

```
[shaftSpeed, shaftTorque, ~] = hev_drivetrain(vehSpd, vehAcc, veh);     % required
shaft speed and torque of the e-motor
shaftPower = shaftTorque * shaftSpeed;                                  % [W] power
at e-motor shaft

% Evaluation of electric motor efficiency corresponding to the motor operating
point through motor map
motSpeed = veh.mot.effMap.GridVectors{1, 1} * 30/pi;      % [rpm]
motTorque = veh.mot.effMap.GridVectors{1, 2};             % [Nm]
motEfficiency = veh.mot.effMap.Values;                    % [-]
motEfficiencyOP = interp2(motSpeed, motTorque, motEfficiency', shaftSpeed * 30/pi,
shaftTorque, 'linear');       % motor efficiency value corresponding to shaft speed
and torque

% Computation of the power demand, i.e. the electric power at motor input (battery
side)
if shaftTorque >= 0
    motPower = shaftPower * 1/motEfficiencyOP;        % if vehicle accelerates, the
motor works in motoring
else
    motPower = shaftPower * motEfficiencyOP;          % if vehicle brakes, the
motor works in regenerative braking
end

% Definition of optimal and maximum engine power operating points -> optimal engine
operating point = min bsfc (max efficiency), maximum power operating point = max
power

% Optimal engine power
[~, minbsfc_index] = min(veh.eng.bsfcMap.Values(:));
```

```
[rowMinbsfc, colMinbsfc] = ind2sub(size(veh.eng.bsfcMap.Values),
minbsfc_index);              % row and column indexes corresponding to min bsfc

engTorqueOpt = veh.eng.bsfcMap.GridVectors{1,2}(colMinbsfc);                %
[Nm]      % engine torque corresponding to min bsfc
engSpeedOpt = veh.eng.bsfcMap.GridVectors{1,1}(rowMinbsfc) * 30/pi;         %
[rpm]     % engine speed corresponding to min bsfc
engPowerOpt = engTorqueOpt * engSpeedOpt * pi/30;                           %
[W]       % optimal engine power corresponding to min bsfc

% Maximum engine power
engSpeed = veh.eng.maxTrq.GridVectors{1,1} * 30/pi;         % [rpm]
engTorque = veh.eng.maxTrq.Values;                          % [Nm]
engPower = engTorque .* engSpeed * pi/30;                   % [W]       % maximum
power profile

[engPowerMax, engPowerMax_index] = max(engPower);           % [max power value,
index corresponding to max power]
engSpeedMaxPower = engSpeed(engPowerMax_index, 1);          % [rpm]     % engine
speed corresponding to max power
engTorqueMaxPower = engTorque(engPowerMax_index, 1);        % [Nm]      % engine
torque corresponding to max power

effGen = 0.89;  % we cannot evaluate the efficiency in advance without knowing the
engine operating point and so the generator speed and torque

if engState
   if  motPower <= effGen * engPowerOpt
       engPowerCmd = engPowerOpt;           % if engine is ON AND the optimal
engine power is sufficient to charge the battery, the engine power is set to the
optimal value
   else
       engPowerCmd = engPowerMax;           % if engine is ON BUT the optimal
engine power is NOT sufficient to charge the battery, the engine power is set to
the max value
   end
else
    engPowerCmd = 0;                        % if engine is OFF, required engine
power set to 0
end
```

Phase 3: add limits on the battery power

```
% evaluation of the maximum and minimum battery power for the considered SOC
battPowerMax = interp1(veh.batt.maxPwr.GridVectors{1,1}, veh.batt.maxPwr.Values,
SOC, 'linear');        % interpolation between the given SOC-MaxPower points
battPowerMin = interp1(veh.batt.minPwr.GridVectors{1,1}, veh.batt.minPwr.Values,
SOC, 'linear');        % interpolation between the given SOC-MinPower points
```

```
engPowerCmd = max(engPowerCmd, (motPower - battPowerMax)/effGen);       %
implementation of first constraint

engPowerCmd = min(engPowerCmd, (motPower - battPowerMin)/effGen);       %
implementation of second constraint

engPowerCmd = max(engPowerCmd, 0);                                      %
implementation of third constraint
```

Phase 4: choose engine speed and engine torque

```
if engPowerCmd == engPowerOpt            % speed and torque values at optimal
engine power
    engSpd = engSpeedOpt * pi/30;        % [rad/s]
    engTrq = engTorqueOpt;               % [Nm]

elseif engPowerCmd == engPowerMax        % speed and torque values at max engine
power
    engSpd = engSpeedMaxPower * pi/30;   % [rad/s]
    engTrq = engTorqueMaxPower;          % [Nm]

elseif engPowerCmd == 0                  % speed and torque values at zero engine
power
    engSpd = 0;
    engTrq = 0;
```

Up to now the controller behaves as the first version. The following part differs from the engine operating point implementation when saturation occurs.

The reason for choosing this new function lies in the need to optimize engine performance while improving fuel efficiency. By adjusting the operating points, the engine can move towards regions with lower brake-specific fuel consumption, leading to a more efficient use of fuel. A lower fuel economy may result from these changes, as the engine shifts to lower *bsfc* values, meaning that less fuel is required to generate the same amount of power. This approach ensures that the engine operates within an optimal efficiency range while respecting torque limits, ultimately enhancing the overall energy management strategy.

```
else         % if the engine power command is different from optimal, maximum and
null set speed and torque trying to stay on OOL

    % Interpolation to find OOL
    engOOLSpeed = veh.eng.oolTrq.GridVectors{1,1};      % [rad/s]
    engOOLTorque = veh.eng.oolTrq.Values;               % [Nm]
    engOOLPower = engOOLTorque .* engOOLSpeed;          % [W]

    engSpd = interp1(engOOLPower, engOOLSpeed, engPowerCmd, 'linear');      %
engine speed on OOL
    engSpd = max(engSpd, veh.eng.idleSpd);                                  %
saturation to avoid engine speed lower than the idle one
    engTrq = engPowerCmd / engSpd;                                          %
compute engine torque from speed and saturated power command
```

```
    % check if the engine torque computed exceed the limit
    engMaxTorqueAllowed = interp1(veh.eng.maxTrq.GridVectors{1,1},
veh.eng.maxTrq.Values, engSpd, 'linear');        % max torque for the chosen engine
speed

    if engTrq > engMaxTorqueAllowed                % if torque exceeds the limit
       engSpd = engSpeedMaxPower * pi/30;          % set the engine speed
corresponding to the max power
       engTrq = engPowerCmd/engSpd;                % and compute again the torque
    end
end

end
```